

Titan Development Doctrine (Constitution)

Purpose

This doctrine is the single source of truth for how developers and AI agents build inside TitanZero/MagicAI.

It defines:

where code must be saved

how data must be structured

how engines interact

how signals and automation work

how AI governance operates

how extensions integrate safely

If a feature violates this doctrine, it is considered non-compliant.

1) The Titan Layer Model

All work must map to one of these layers:

Interface Layer → UI and user workflows

System Layer → feature bundles (CRM, QC, Dashboards)

Engine Layer → domain logic (Work, Money, Talk, Signal, Pulse)

Data Core Layer → tz_* tables + event history

Signal Layer → append-only event stream

Automation Layer → rule execution (Pulse)

AI Layer → assistants + governance (Zero)

Extension Layer → optional modules (ext_*)

Analytics Layer → metrics derived from signals

No layer may skip the one below it.

2) Where Developers Save Code (Canonical)

This is the wiring of your current codebase.

Engines (domain logic)

app/Services/Titan<Function>Engine/

Systems (feature bundles like CRM)

app/Services/Titan<Thing>System/

Domain helpers (optional, clean separation)

app/Domains/<Thing>/

Controllers (thin)

app/Http/Controllers/<Area>/

Livewire components (if used)

app/Livewire/<Thing>/

Views

resources/views/panel/<thing>/

Routes

routes/panel.php (admin/panel UI)

routes/api.php (PWA/mobile endpoints)

routes/web.php (public pages)

Background jobs

app/Jobs/<Thing>/

Providers (boot/bindings only)

app/Providers/

3) Data Prefix Doctrine (Non-negotiable)

(no prefix) → platform/system tables

tz_* → core operational business data

ext_* → extension-owned tables

Examples:

users, teams, menus = platform

tz_jobs, tz_invoices, tz_messages = core

ext_social_posts, ext_campaigns = extension

4) Tenant Isolation Doctrine

Tenant key is:

team_id

Rules:

every tz_* and ext_* table must include team_id

every query must scope by team_id

every engine method must enforce tenant scope

AI, automation, signals, analytics must all be tenant-scoped

Any code that reads/writes tenant data without tenant scoping is a critical defect.

5) Engine Ownership Doctrine

Each engine owns its domain and is the only writer of its tables.

Examples:

Work Engine owns tz_jobs*, tz_tasks*, tz_visits*, tz_checklists*

Money Engine owns tz_quotes*, tz_invoices*, tz_payments*

Talk Engine owns tz_threads*, tz_messages*, tz_contacts*

Evidence Engine owns tz_evidence_*

Docs Engine owns tz_documents*

Signal Engine owns tz_signals*

Pulse Engine owns tz_automation_*

Analytics Engine owns tz_metrics*

Zero Engine owns governance + AI log/artifact tables

No engine may directly mutate another engine's domain tables.

6) The Data Core Table Pattern

Every major entity should follow this shape:

tz_<entity>
tz_<entity>_events
tz_<entity>_notes
tz_<entity>_links
tz_<entity>_evidence

Example for jobs:

tz_jobs
tz_job_events
tz_job_notes

tz_job_links
tz_job_evidence

This ensures:

state is clean (core table)

history is preserved (events)

compliance is supported (evidence)

relationships are traceable (links)

7) Signal Doctrine (Event Language)

All meaningful actions emit signals.

Signal naming format:

domain.entity.event

Examples:

work.job.created

work.job.completed

finance.invoice.overdue

talk.message.received

Signal properties (minimum):

team_id

event_type

entity_type

entity_id

payload

timestamp

Signals are:

append-only

immutable

replayable

Signals are stored by Titan Signal Engine in:

tz_signals

tz_signal_events

tz_signal_consumers

8) Automation Doctrine (Titan Pulse)

Pulse is the only automation executor.

Pulse:

subscribes to signals

evaluates rules

executes actions through engine APIs

logs outcomes

Rules must define:

trigger signal

conditions

actions

priority

cooldown

Pulse writes only:

tz_automation_*

Pulse never writes operational tables directly.

9) AI Doctrine (Titan Zero)

Titan Zero is the governance layer and overseer.

It includes:

TitanZeroCore (synthesis)

OmegaCore (strategic planning)

OmicronCore (detail validation)

Normal mode:

Titan Zero Mode

High-risk mode:

Trinary Bayesian Mode (Omega + Omicron + Core)

Titan Zero:

proposes actions

evaluates risk

approves/rejects

generates artifacts

routes models via multimodal router

Titan Zero never writes operational state in tz_*. It writes only governance + AI metadata tables.

10) Risk & Approval Doctrine

Risk levels:

LOW → Pulse can execute automatically

MEDIUM → optional review / bounded

HIGH → Titan Zero approval required

CRITICAL → Titan Zero + human approval required

All approvals are visible in Titan Command governance queue.

11) System/Module Doctrine (CRM example)

A “system” (CRM, QC, Marketing) is a composition layer.

CRM composes:

Customer Engine

Talk Engine

Work Engine

Money Engine

Rule:

Systems do not “own” engine tables

Systems call engines through service methods

Systems provide UI workflows and dashboards

Systems live in:

app/Services/Titan<Thing>System/

12) Interface Doctrine (Command / Talk / Origin / Go)

Interfaces are views + endpoints only.

Interfaces:

call engine APIs

never write DB directly

never embed core logic in controllers/views

Titan Go modes:

Field Mode

Command Mode

Dispatch Mode

QC Mode

All modes still call the same engine services.

13) Extension Doctrine (Marketplace safe plug-ins)

Extensions:

live in app/Extensions/<ExtensionName>/ (if using extension system)

store data in ext_*

integrate via signals + engine APIs

do not modify core engines

Extensions must have:

manifest

provider

routes/views isolated

SQL-first install scripts (preferred)

14) What “Done” Means (Definition of Done)

A feature is “done” only when it includes:

1. Code placed in correct folders (Services/Controllers/Views)
2. Tenant scoping enforced (team_id)

3. Tables follow prefix + entity pattern
4. Signals emitted for lifecycle changes
5. Automation rules defined (if applicable)
6. Governance risk classification (if applicable)
7. Audit trail preserved (events/logs)
8. UI integrated into Titan Command/Go/Origin as appropriate

15) Canonical Naming

Engines:

Titan + Function + Engine

Systems:

Titan + Thing + System

Signals:

domain.entity.event

Tables:

tz_<entity>

ext_<entity>

